

Kubernetes + Monitoring

Siddhesh Prabhugaonkar

Microsoft Certified Trainer

siddheshpg@azureauthority.in

Kubernetes introduction, architecture, YAML

Deploy to EKS

Introduction to Monitoring

Prometheus, Grafana

Using Prometheus & Grafana for system metrics and alerting

Section 1 — What is Kubernetes?

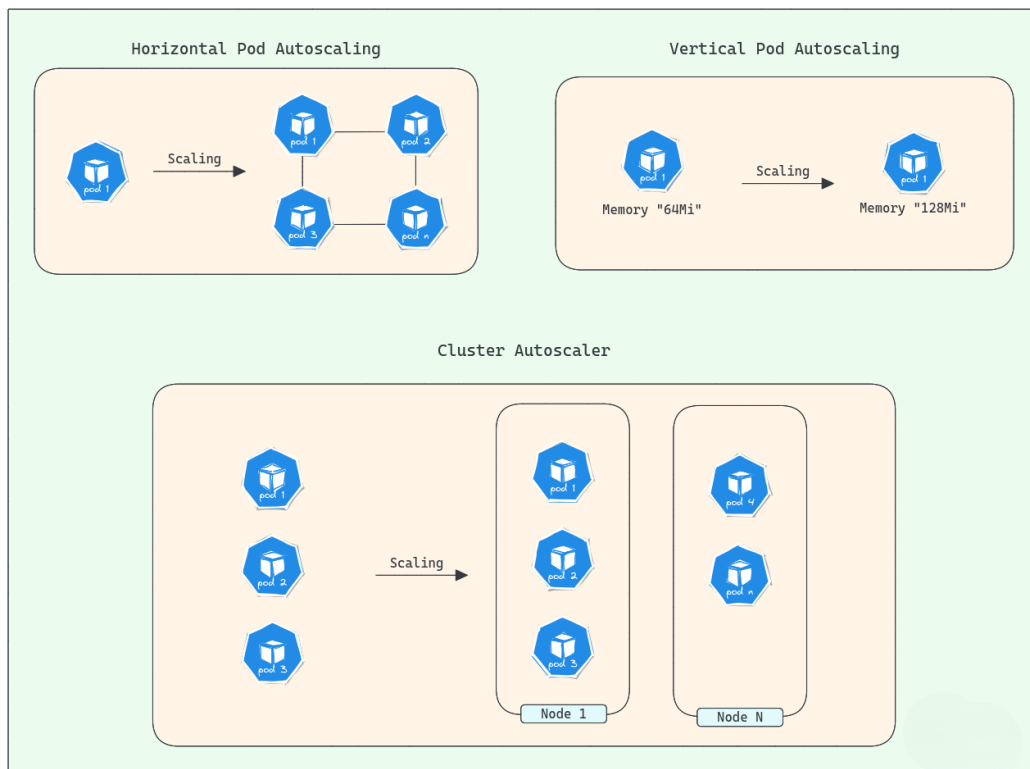
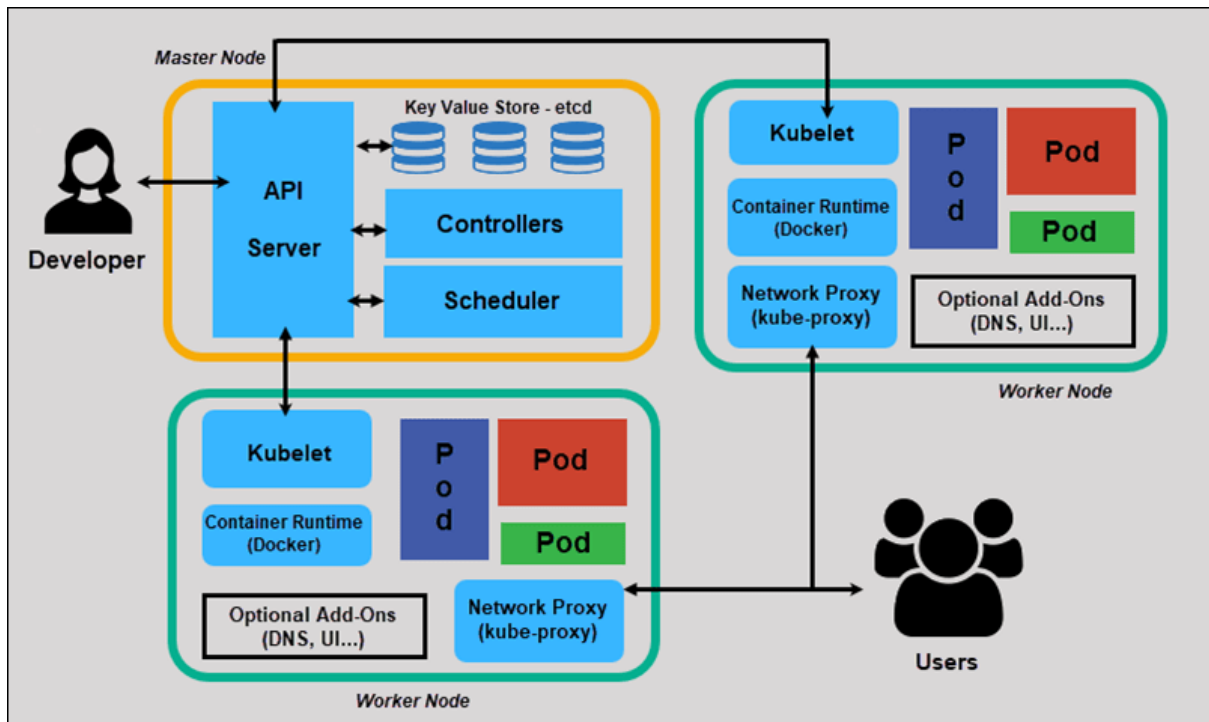
Kubernetes is a **container orchestration platform** used to run and manage containerized applications at scale. It ensures your app:

- keeps running even if containers crash
- can scale up/down
- receives updates without downtime
- has stable networking and DNS
- can mount storage
- has monitoring hooks

Modern applications are split into independent microservices (React frontend, NestJS API, Oracle DB). Managing multiple services manually becomes complex. Kubernetes solves this by providing:

- **Automatic scaling**
 - **Self-healing pods** (restart failed containers)
 - **Rolling updates & rollbacks**
 - **Declarative deployments** via YAML
 - **Service discovery and stable networking**
 - **Portable workloads** across clouds
-

Section 2 — Kubernetes Architecture



Section 3 — Understanding Kubernetes YAML

Kubernetes uses **YAML files** to define the desired state of applications.

Basic YAML Rules

- Use **spaces**, not tabs
- Indentation defines structure
- Key-value format
- Lists start with –

Examples

- `key: value` `# Simple key-value`
- `list:`
 - `item1`
 - `item2`
- `nested:`
 - `level1:`
 - `level2: value`
- Indentation matters (use spaces, not tabs).

Required Sections in Kubernetes YAML

- `apiVersion`: API version of the resource.
- `kind`: Type of Kubernetes resource (e.g., `Deployment`).
- `metadata`: Metadata like name and labels.
- `spec`: Specification of desired state (replicas, containers, etc.).

Kubernetes Resource Types

- **Pod** – smallest deployable unit
 - **Deployment** – manages replica Pods, handles updates
 - **Service** – exposes Pods inside/outside cluster
 - **ConfigMap** – non-secret configs
 - **Secret** – sensitive data
 - **Ingress** – URL routing (not used here)
 - **StatefulSet** – for databases
 - **PersistentVolumeClaim** – storage
-

Section 4 — Kubernetes YAML for EMS Case Study

EMS consists of three containers:

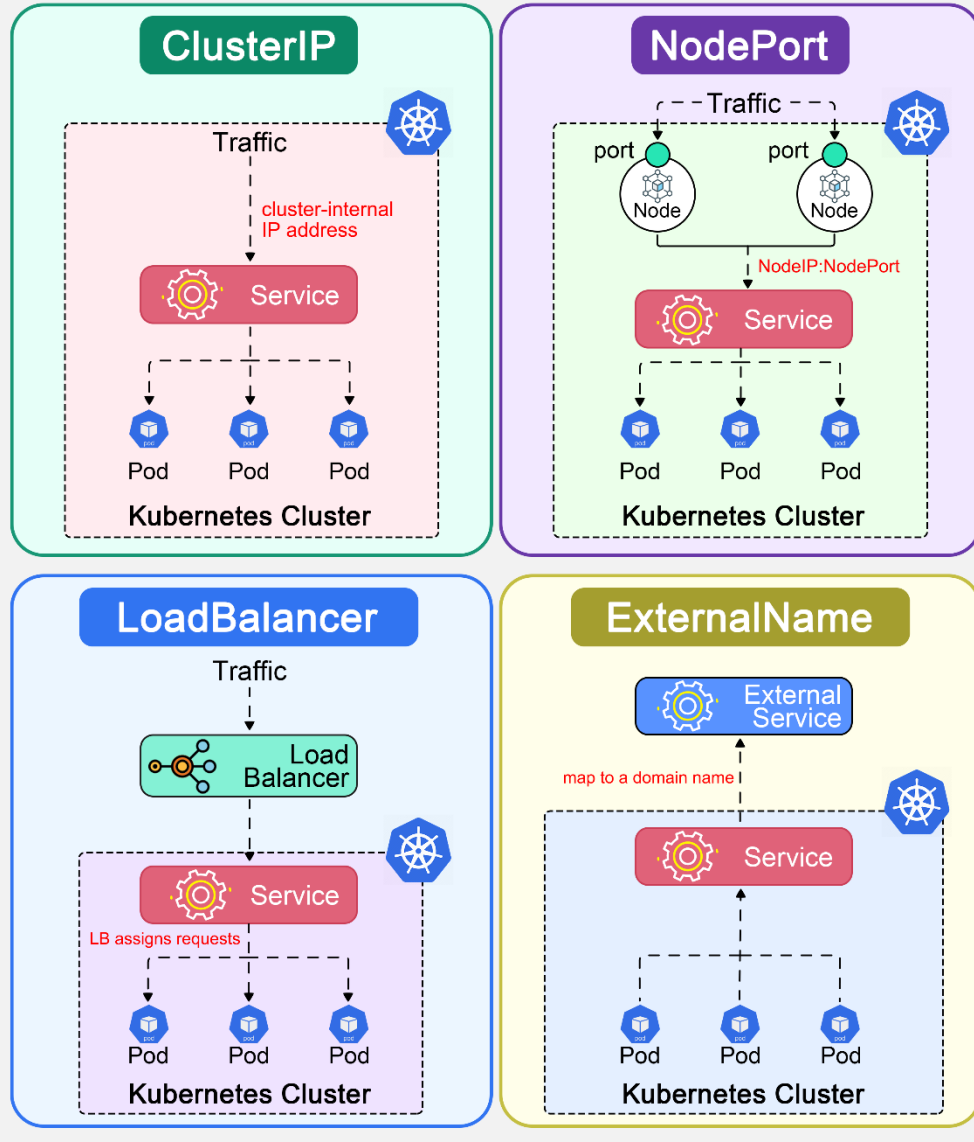
1. **React Frontend** (nginx)
2. **NestJS API**
3. **Oracle XE DB**

Images come from **Amazon ECR**.

No ALB is used — only LoadBalancer Services where needed.

4 K8S Service Types

ByteByteGo



Kubernetes kind types

Kind	Meaning / Purpose
Pod	The smallest deployable unit in Kubernetes. A Pod runs one or more containers that share the same network and storage context. All apps ultimately run inside Pods.
Deployment	A controller that manages stateless Pods. Ensures the correct number of Pods are running, enables rolling updates , rollbacks , and easy scaling . Most common way to run applications.

Kind	Meaning / Purpose
ReplicaSet	Maintains a stable number of identical Pods. Deployments use ReplicaSets underneath. Rarely created manually — usually created automatically by Deployments.
Service	Provides a stable network endpoint (cluster IP + port) for connecting to Pods. Used for load-balancing and service discovery. Even if Pods restart, the Service address stays the same.
ConfigMap	Stores non-sensitive configuration data such as URLs, feature flags, environment settings. Mounted into Pods via environment variables or files.
Secret	Stores sensitive data such as passwords, API tokens, certificates. Encoded using Base64. Mountable into Pods as env variables or files.
PersistentVolume (PV)	Represents a piece of storage provisioned in the cluster (EBS, NFS, local disk). A cluster-level resource.
Namespace	Logical grouping/partition inside a cluster. Helps organize applications (e.g., dev, test, prod) and apply access controls or quotas.
Node	A physical or virtual machine where Kubernetes runs Pods (EC2 instance, on-prem VM, or Fargate virtual node). Contains kubelet, container runtime, kube-proxy.
Prometheus (CRD)	A Custom Resource Definition installed by the Prometheus Operator. Allows Kubernetes to manage a Prometheus server instance declaratively (scraping rules, retention, config).
GrafanaDashboard (CRD)	A CRD installed by Grafana Operator or kube-prometheus-stack. Lets you define Grafana dashboards as YAML so they are versioned and deployed through Kubernetes.
VerticalPodAutoscaler (VPA)	Automatically adjusts CPU and Memory requests/limits for Pods based on usage. Ensures optimal resource allocation. Complements or replaces HPA for certain workloads.

4.1 Namespace

```
apiVersion: v1
kind: Namespace
metadata:
  name: ems
```

4.2 Secrets

```
apiVersion: v1
kind: Secret
metadata:
```

```
name: ems-db-secret
namespace: ems
type: Opaque
stringData:
  username: training
  password: training123
```

4.3 Frontend Deployment + Service

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ems-frontend
  namespace: ems
spec:
  replicas: 2
  selector:
    matchLabels:
      app: ems-frontend
  template:
    metadata:
      labels:
        app: ems-frontend
    spec:
      containers:
        - name: frontend
          image: <ECR_URL>/ems-frontend:latest
          ports:
            - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: ems-frontend-svc
  namespace: ems
spec:
  type: LoadBalancer
  selector:
    app: ems-frontend
  ports:
    - port: 80
      targetPort: 80
```

4.4 API Deployment + Service

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ems-api
  namespace: ems
spec:
  replicas: 2
  selector:
    matchLabels:
```

```

    app: ems-api
template:
  metadata:
    labels:
      app: ems-api
  spec:
    containers:
      - name: api
        image: <ECR_URL>/ems-api:latest
        ports:
          - containerPort: 3000
        env:
          - name: DB_HOST
            value: "oracledb"
          - name: DB_PORT
            value: "1521"
          - name: DB_USERNAME
            valueFrom:
              secretKeyRef:
                name: ems-db-secret
                key: username
          - name: DB_PASSWORD
            valueFrom:
              secretKeyRef:
                name: ems-db-secret
                key: password
---
apiVersion: v1
kind: Service
metadata:
  name: ems-api-svc
  namespace: ems
spec:
  type: ClusterIP
  selector:
    app: ems-api
  ports:
    - port: 3000
      targetPort: 3000

```

4.5 Oracle DB StatefulSet

```

apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: oracledb
  namespace: ems
spec:
  serviceName: oracledb
  replicas: 1
  selector:
    matchLabels:
      app: oracledb
  template:
    metadata:
      labels:
        app: oracledb

```



```
spec:
  containers:
    - name: db
      image: container-registry.oracle.com/database/express:21.3.0-xe
      ports:
        - containerPort: 1521
      env:
        - name: ORACLE_PWD
          value: "OraclePassword123"
      volumeMounts:
        - name: oradata
          mountPath: /opt/oracle/oradata
  volumeClaimTemplates:
    - metadata:
        name: oradata
      spec:
        accessModes: [ "ReadWriteOnce" ]
        resources:
          requests:
            storage: 20Gi
```

Section 5 — Helm

What is Helm?

Helm = package manager for Kubernetes

Similar to apt, npm, pip

You install entire apps with a **single command** using “charts”

Example Helm install:

```
helm install monitoring prometheus-community/kube-prometheus-stack \
  -n monitoring --create-namespace
```

Why Helm?

- Simplifies Kubernetes deployments
 - Templates values
 - Version control
 - Used heavily in real-world EKS setups
-

Section 6 — Deploying EMS to EKS

Flow for class:

1. pre-creates ECR repositories
2. uploads images to ECR
3. create an EKS cluster using **AWS Console**
4. create EKS Nodes or Fargate Profile using console
5. use **Podman Desktop** → **Kubernetes tab** to connect to EKS
6. apply YAML via Podman Desktop
7. verify deployment
8. test LoadBalancer URL for frontend

Using Podman Desktop (macOS)

1. Open Podman Desktop
2. Go to **Kubernetes** section
3. Add cluster → choose **AWS EKS**
4. Podman Desktop downloads and configures `kubeconfig`
5. Students can:
 - Apply YAML
 - View Pods, Deployments
 - Inspect logs

Section 7 — Monitoring

7.1 Prometheus Overview

- Collects cluster metrics
 - Scrapes endpoints
 - Stores time-series data
-

7.2 Grafana Overview

- Visual dashboards
- Alerts
- Supports Prometheus, CloudWatch

7.3 Installing Prometheus & Grafana via Helm

Students can install via Podman Desktop's Helm UI (no CLI required):

Steps:

1. Podman Desktop → Kubernetes → Helm Charts
 2. Add repo:
 - o prometheus-community
 3. Install **kube-prometheus-stack** chart
 4. Access Grafana via port-forward UI (Podman Desktop has a port-forward tool)
-

Section 8 — CloudWatch Container Insights

CloudWatch Container Insights provides:

1. Performance Metrics

- CPU / Memory usage per pod
- Node CPU / Memory
- Volume usage
- Network traffic

2. Logs

- Container stdout/stderr
- Application logs
- Kubernetes system component logs

3. Automatic Dashboards

For:

- EKS
- ECS
- Fargate

4. Metrics collected

Cluster level:

- node_cpu_utilization
- node_memory_utilization
- pod_count
- container_count

Pod level:

- pod_cpu_utilization
- pod_memory_usage
- pod_network_rx
- pod_network_tx

Control plane logs:

- API server
- Kubelet
- Scheduler

Why CloudWatch is essential in this setup

- Students won't configure full Prometheus exporters
- CloudWatch works automatically with ECS Fargate
- Dashboard templates help students understand workloads
- Logs integrate with Lambda, SNS, S3
- CloudWatch alarms can replace Alertmanager

How to access it

Inside AWS Console:

- Go to **CloudWatch**
- Select **Container Insights**
- Choose **EKS Cluster** or **ECS Cluster**
- Explore dashboards
- View node/pod metrics
- View application logs
- Create alerts
- Create anomaly detection

Section 9 — Production Path: ECR + ECS Fargate

Why Fargate?

- No servers
- No node patching
- Automatic scaling
- Works seamlessly with CloudWatch
- Perfect for EMS production workloads

Flow

1. Images in ECR
2. ECS Task Definition
3. ECS Service
4. Network → VPC + Subnets
5. Application Load Balancer (optional)
6. CloudWatch Insights

Only DB is external (Oracle on EC2 or RDS).

Section 10 — Consolidated Commands

Students won't run these.

```
# Build
docker build -t ems-api .
docker build -t ems-frontend .

# Tag
docker tag ems-api <ECR_URL>/ems-api:latest
docker tag ems-frontend <ECR_URL>/ems-frontend:latest

# Push
docker push <ECR_URL>/ems-api:latest
docker push <ECR_URL>/ems-frontend:latest

# Helm repo
helm repo add prometheus-community https://prometheus-
community.github.io/helm-charts
helm repo update
```

Section 11 — Summary

- Kubernetes is a declarative system using YAML
- Students learn Deployments, Services, StatefulSets
- EMS is deployed to **EKS** for learning
- Production uses **ECR + ECS Fargate**
- Podman Desktop (macOS) replaces AWS CLI
- Monitoring uses **Prometheus + Grafana** for K8s learning

- **CloudWatch Container Insights** is the main recommended monitoring for production

kubectl Commands & Their Meanings

Basic Get Commands

Command	Meaning
<code>kubectl get nodes</code>	List all nodes in the cluster.
<code>kubectl get pods</code>	List all pods in the current namespace.
<code>kubectl get pods -A</code>	List pods across all namespaces.
<code>kubectl get svc</code>	List services in the namespace.
<code>kubectl get deploy</code>	List deployments.
<code>kubectl get rs</code>	List ReplicaSets.
<code>kubectl get cm</code>	List ConfigMaps.
<code>kubectl get secret</code>	List Secrets.
<code>kubectl get events</code>	View recent events (often used to debug).
<code>kubectl get ingress</code>	List Ingress resources.
<code>kubectl get pv</code>	List PersistentVolumes.
<code>kubectl get pvc</code>	List PersistentVolumeClaims.

Describe & Inspect

Command	Meaning
<code>kubectl describe pod <name></code>	Detailed information about a pod.
<code>kubectl describe node <name></code>	Detailed information about a node.
<code>kubectl logs <pod></code>	Fetch logs from a pod.
<code>kubectl logs <pod> -f</code>	Stream logs (follow).
<code>kubectl exec -it <pod> -- bash</code>	Exec into a pod shell.
<code>kubectl top pod</code>	Show CPU/memory usage of pods (metrics server needed).
<code>kubectl top node</code>	Show node resource usage.

Create, Apply, Delete

Command	Meaning
<code>kubectl create -f file.yaml</code>	Create resources from a manifest file.
<code>kubectl apply -f file.yaml</code>	Apply changes (create/update) using declarative approach.

Command	Meaning
<code>kubectl delete -f file.yaml</code>	Delete resources from a manifest.
<code>kubectl delete pod <name></code>	Delete a specific pod (restarts via Deployment).
<code>kubectl delete ns <name></code>	Delete a namespace.

Deployment Management

Command	Meaning
<code>kubectl rollout status deploy/<name></code>	Check rollout status.
<code>kubectl rollout restart deploy/<name></code>	Restart a Deployment.
<code>kubectl rollout undo deploy/<name></code>	Roll back to previous version.
<code>kubectl scale deploy/<name> --replicas=3</code>	Scale a deployment to 3 replicas.
<code>kubectl set image deploy/<name> container=image:tag</code>	Update container image in a deployment.

Debugging Tools

Command	Meaning
<code>kubectl describe pod <name></code>	Check events, state, scheduling issues.
<code>kubectl get events --sort-by=.metadata.creationTimestamp</code>	View events chronologically.
<code>kubectl exec</code>	Check inside containers.
<code>kubectl port-forward pod/<name> 8080:80</code>	Forward port to access a service locally.
<code>kubectl cp <pod>:/path /localpath</code>	Copy files from a pod.
<code>kubectl debug pod/<name> --image=busybox</code>	Create an ephemeral debug container (K8s 1.23+).

Namespace Management

Command	Meaning
<code>kubectl get ns</code>	List namespaces.
<code>kubectl config set-context --current --namespace=<ns></code>	Set default namespace.
<code>kubectl create ns <name></code>	Create a namespace.

Cluster Configuration

Command	Meaning
<code>kubectl config get-contexts</code>	List configured contexts.
<code>kubectl config use-context <name></code>	Switch kubeconfig context.
<code>kubectl config view</code>	Show kubeconfig details.
<code>kubectl version</code>	Show client & server versions.

Working with YAML

Command	Meaning
<code>kubectl explain pod</code>	Show API documentation for Pod.
<code>kubectl explain deploy.spec</code>	Inspect schema for fields.
<code>kubectl get deploy <name> -o yaml</code>	Get manifest of a live resource.
<code>kubectl apply -k <kustomize_dir></code>	Apply using Kustomize.

Cleaning Up

Command	Meaning
<code>kubectl delete pod --all</code>	Delete all pods in namespace.
<code>kubectl delete all --all</code>	Delete everything in namespace (use cautiously).

Shortcuts (Aliases)

Shortcut	Meaning
<code>po</code>	pods
<code>svc</code>	service
<code>deploy</code>	deployment
<code>rs</code>	replicaset
<code>cm</code>	configmap
<code>ns</code>	namespace

Example:

```
kubectl get po == kubectl get pods
```


How to Connect to EKS from Podman Desktop (via kubectl)

Podman Desktop itself does **not** connect directly to EKS.

It simply reads your local **kubeconfig** (usually `~/.kube/config`) and displays the cluster + pods in the UI.

So the real task is:

Get kubeconfig from EKS → Ensure kubectl works → Podman Desktop automatically shows the cluster.

Step 1: Install Required Tools

✓ kubectl

✓ awscli

✓ Podman Desktop (already installed)

Verify:

```
aws --version
kubectl version --client
```

Step 2: Configure AWS Credentials

Login to AWS using:

```
aws configure
```

Provide:

- AWS Access Key
- Secret Key
- Region (e.g., ap-south-1, us-east-1)
- Output format (json)

Verify:

```
aws sts get-caller-identity
```

Step 3: Update kubeconfig for EKS

Run this command to fetch cluster access info and merge it into your ~/.kube/config:

```
aws eks update-kubeconfig --name <cluster-name> --region <region>
```

Example:

```
aws eks update-kubeconfig --name my-eks --region ap-south-1
```

What it does:

- Fetches cluster API endpoint
- Gets cluster certificate
- Creates/updates kubeconfig file
- Sets correct authentication via AWS IAM auth

Verify:

```
kubectl get nodes
```

If you see nodes → **your kubectl is connected to EKS successfully.**

Step 4: Open Podman Desktop

Go to:

Podman Desktop → Kubernetes → Contexts / Clusters

You will see your EKS cluster listed.

Click:

- ✓ **Activate Context** (if not active)
- ✓ You can now view Pods, Deployments, Logs, Events

No extra configuration is required in Podman Desktop.